OpenZeppelin | security

OpenZeppelin SUI Library Audit

March 2, 2026

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5
- access 5
- math 6

Security Model and Trust Assumptions 6
- Privileged Roles 7

High Severity 8
- H-01 Multiplication and Modulo Operation Returns Incorrect Results on Overflow 8

Low Severity 9
- L-01 Inaccurate Data Emitted in Events 9
- L-02 Reused OwnershipTransferred Event in unwrap 9

Notes & Additional Information 10
- N-01 Missing Event Emission on State Change 10
- N-02 Function Visibility Overly Permissive 11
- N-03 Code Convention Improvements 11
- N-04 Missing Documentation 12
- N-05 Code and Documentation Inconsistencies 12
- N-06 Incorrect Documentation 13
- N-07 Inconsistent Error Handling in Mathematical Helper Functions 14
- N-08 Unhandled Overflow in safe_downcast_balance 15
- N-09 Multiple Evaluations of Macro Parameters 15
- N-10 Unnecessary Option Destruction 16

Conclusion 17

Appendix 18
- Issue Classification 18

Summary

Type	Library
Timeline	From 2026-01-26 To 2026-02-06
Languages	Move

Total Issues	13 (10 resolved, 1 partially resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	1 (1 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	2 (2 resolved)
Notes & Additional Information	10 (7 resolved, 1 partially resolved)

Scope

OpenZeppelin audited the OpenZeppelin/contracts-sui repository at commit 30de819.

In scope were the following files:

```
├── contracts
│   └── access
│       └── sources
│           └── ownership_transfer
│               ├── delayed.move
│               └── two_step.move
└── math
    └── core
        └── sources
            ├── internal
            │   ├── common.move
            │   └── macros.move
            ├── decimal_scaling.move
            ├── rounding.move
            ├── u8.move
            ├── u16.move
            ├── u32.move
            ├── u64.move
            ├── u128.move
            ├── u256.move
            └── u512.move
```

System Overview

This security audit evaluated select components of the OpenZeppelin SUI library, a collection of foundational modules designed to facilitate the development of secure and reliable packages on the SUI blockchain. The library aims to mirror the behavior and security guarantees of the widely adopted OpenZeppelin Solidity library on Ethereum, while adapting its design to the Move programming language and SUI's object-centric execution model.

The scope of this audit covered two core components of the library: `access` and `math`. These components provide critical primitives that are expected to be reused across governance systems, privileged role management, and financial logic, making their correctness and robustness essential to downstream applications.

`access`

This component introduces two wrapper-based access-control mechanisms for the capabilities. Both wrappers expose immutable and mutable borrow functions, as well as a hot-potato `borrow_val`/`return_val` pattern, allowing the wrapped capability to be used without unwrapping it.

The `delayed` wrapper enables time-delayed transfers or unwrapping of a capability, providing external observers and stakeholders with a minimum reaction window before sensitive actions are finalized. Its flow operates as follows:

1. The owner wraps a capability with a minimum delay parameter.
2. The owner schedules either a transfer to a new address or an unwrap operation.
3. After the configured delay has elapsed, the owner executes the scheduled action.
4. At any point before execution, the owner may cancel the pending schedule.

The `two_step` wrapper enforces a two-phase ownership transfer process, designed to mitigate irreversible human errors such as accidentally transferring a capability to an incorrect address. Its flow operates as follows:

1. A prospective recipient submits a transfer request specifying the target wrapper, which is sent to the current owner.

2. The current owner may accept the request, finalizing the transfer, or reject it.
    - Alternatively, the owner may directly unwrap the capability at any time without requiring a request.

`math`

This component provides a comprehensive suite of arithmetic utilities for Move's standard unsigned integer types, including `u8`, `u16`, `u32`, `u64`, `u128`, and `u256`. Exposed operations include, among others, averaging, checked bit shifting, multiply-then-divide, multiply-then-shift-right, logarithmic functions (base 2, 10, and 256), square roots, modular arithmetic, and modular inverse. In addition, the library defines a custom `u512` type to support higher-precision arithmetic, along with constructors, accessors, comparison helpers, and arithmetic operations involving `u256` values. Supporting modules define explicit rounding behaviors for arithmetic operations and provide decimal scaling utilities, enabling safe conversion between token amounts with differing decimal representations, particularly relevant for cross-chain or multi-asset systems.

Security Model and Trust Assumptions

The security model of the audited components relies on capability-based access control and explicit user interactions enforced at the module level. Correct behavior assumes that capabilities are held and managed by trusted entities and that users interacting with these modules understand the implications of wrapping, transferring, and unwrapping capabilities.

The `two_step` wrapper intentionally diverges from the OpenZeppelin Solidity implementation: instead of the current owner initiating a transfer for the prospective owner to accept, the SUI version allows any third party to submit a transfer request that the current owner must then accept or reject. While the owner retains full authority over whether a transfer is finalized, this inversion means they must handle unsolicited incoming requests. Since any address can submit requests, the model is susceptible to request spamming, which could lead to operational overhead or degraded user experience. Integrators are expected to account for this behavior and, if necessary, implement mitigations such as off-chain filtering or additional validation logic.

The `delayed` wrapper assumes that the configured time delay is sufficient to allow monitoring parties (e.g., governance participants or investors) to observe and react to scheduled actions before execution. The effectiveness of this mechanism depends on the correctness of the time-tracking logic and on users actively monitoring scheduled operations.

Privileged Roles

Privileged actions within the audited modules are strictly tied to possession of the relevant capability.

In the `delayed` module, only the holder of the wrapped capability is authorized to schedule a transfer or an unwrap operation. Likewise, only the capability holder may execute the action once the required delay has elapsed. No external party can bypass the delay or force execution without control of the underlying capability.

In the `two_step` module, privileges are split across two roles. Any third party may submit a request to receive a wrapped capability, but such a request has no effect unless explicitly accepted by the current capability holder. The authority to approve and finalize a transfer remains exclusively with the owner of the wrapped capability. This model preserves ownership control while allowing flexibility in initiating transfers, but it also places responsibility on the owner to manage and respond to incoming requests appropriately.

High Severity

H-01 Multiplication and Modulo Operation Returns Incorrect Results on Overflow

The `mul_mod_impl` function utilizes the `div_rem_u256` to apply a modulo to the product of operands `a` and `b`. When executing this division, the implementation ignores whether an overflow occurs in the quotient and directly utilizes the remainder. However, when the quotient cannot fit within the `u256` data size, the `div_rem_u256` function is designed to return zero values for both the quotient and the remainder while setting an overflow flag to `true`.

Therefore, the `mul_mod_impl` function will return incorrect results whenever the internal multiplication and subsequent division trigger an overflow. This vulnerability directly impacts the `mul_mod` function and any downstream consumer that calls it with large `u256` operands and a `modulus` small enough for the quotient to exceed `u256`. In practice, this covers any `modulus` below `u256::MAX` when both operands are large, causing `mul_mod` to silently return **0** instead of the correct remainder.

Although `mul_mod_impl` is also called within `inv_mod_extended_impl`, the overflow condition cannot be triggered in that context. The values passed to `mul_mod_impl` from the *Extended Euclidean Algorithm* are inherently bounded. Their product will always be less than `modulus²`, so the resulting quotient is less than `modulus` and always fits within `u256`.

Consider modifying `div_rem_u256` to continue the remainder computation even when the quotient overflows `u256`, since the remainder is always representable regardless of quotient size. Alternatively, consider introducing a dedicated remainder-only function that `mul_mod_impl` can call, avoiding quotient tracking entirely.

Update: *Resolved in pull request #151.*

Low Severity

L-01 Inaccurate Data Emitted in Events

In the `schedule_transfer` and `schedule_unwrap` functions, the caller is permitted to provide the `current_owner` address as an argument. This address is subsequently emitted within the `TransferScheduled` event. However, this design allows the owner of the wrapper to provide arbitrary or incorrect data. Consequently, off-chain components that rely on event logs for tracking ownership or state transitions may be misled by inaccurate information.

Consider retrieving the `current_owner` directly from `ctx.sender()` when emitting the event. This approach ensures that the emitted data is verified and reflects the actual transaction signer, thereby maintaining the integrity of off-chain monitoring systems.

Update: *Resolved in pull request #174.*

L-02 Reused `OwnershipTransferred` Event in `unwrap`

The `unwrap` function of the `delayed` module emits an `OwnershipTransferred` event with `previous_owner: ctx.sender()` and `new_owner: ctx.sender()` (the same address). This is semantically misleading because an unwrap destroys the wrapper and returns the capability to the owner. It is not a transfer. Off-chain systems parsing `OwnershipTransferred` events cannot distinguish between a real transfer and an unwrap without additional context.

The module already defines separate scheduling events (`TransferScheduled` vs `UnwrapScheduled`), making the inconsistency at the execution layer more noticeable. As such, off-chain indexers and analytics tools may miscount transfers or misinterpret unwraps as self-transfers. Governance dashboards tracking ownership history will show a false "transfer to self" entry.

Consider emitting a dedicated event (e.g., `Unwrapped`) instead of reusing `OwnershipTransferred`. This aligns with the scheduling phase which already distinguishes between transfers and unwraps via separate event types.

Update: *Resolved in pull request #168.*

Notes & Additional Information

N-01 Missing Event Emission on State Change

Throughout the codebase, multiple instances of state-changing operations were identified that do not emit events:

- In the `two-step` module, the following operations execute state changing operations without emitting any event:
  - The `wrap` function creates a new wrapper.
  - The `unwrap` function destroys the wrapper and returns the capability.
  - The `reject` function deletes an ownership request. In contrast, the `request` and `transfer` functions properly emit events. This incomplete coverage prevents off-chain systems from tracking when wrappers are created, destroyed outside the transfer flow, or when ownership requests are rejected.
- In the `delayed` module, the `cancel_schedule` function silently cancels a pending transfer or unwrap by extracting and dropping the `PendingTransfer` struct without emitting an event. Therefore, an off-chain observer who previously received a `TransferScheduled` or `UnwrapScheduled` events has no way to learn that the scheduled operation was cancelled.
- In the `delayed` module, the `wrap` function creates a new `DelayedTransferWrapper` with a specified delay but emits no event. While the module emits events for other lifecycle operations, such as `scheduling` or `execution`, off-chain systems are not notified when a new delayed transfer wrapper is created or which delay was configured.

Consider ensuring that every significant state mutation is accompanied by a dedicated event. Otherwise, it can lead to stale or misleading state in indexers, dashboards, and monitoring systems.

Update: *Resolved in pull request #159 and pull request #169.*

N-02 Function Visibility Overly Permissive

Throughout the codebase, multiple instances of functions with unnecessarily permissive visibility were identified:

- The `mul_div_u256_wide` function in `macros.move` with `public(package)` visibility could be limited to `private`.
- The `mul_shr_u256_fast` function in `macros.move` with `public(package)` visibility could be limited to `private`.
- The `mul_shr_u256_wide` function in `macros.move` with `public(package)` visibility could be limited to `private`.
- The `round_division_result` function in `macros.move` with `public(package)` visibility could be limited to `private`.
- The `mod_sub_impl` function in `macros.move` with `public(package)` visibility could be limited to `private`.

To better convey the intended use of functions, consider changing a function's visibility to be only as permissive as required.

Update: *Acknowledged, not resolved. The OpenZeppelin SUI development team stated:*

> *We have chosen to maintain our current test organization with tests in a dedicated `tests/` directory, as this aligns with our project structure and testing conventions. Therefore, we will not be implementing this recommendation at this time.*

N-03 Code Convention Improvements

Throughout the codebase, multiple instances were identified where the implementation does not strictly adhere to Move coding conventions and organizational best practices:

- **Inconsistent Error Code Indexing**: In the `two_step` module, error codes do not begin from a zero-based index. Standardizing error constants to start at 0 (or a consistent base) improves maintainability and predictability for developers interacting with the contract.
- **Suboptimal Argument Ordering**: In the `schedule_transfer` and `schedule_unwrap` functions of the `delayed` module, the `Clock` object is not positioned as the final argument. Objects should go first except for this special `Clock` object as per the code quality checklist.

Adhering to a uniform code style is essential for long-term project health and eases the onboarding process for new contributors.

Consider implementing these changes to properly follow Move coding conventions.

Update: *Resolved in pull request #163.*

N-04 Missing Documentation

Throughout the codebase, multiple instances of missing documentation were identified:

- `u8`, `u16`, `u32`, `u64`, `u128`, `u256`, `macros`, `rounding` and `decimal_scaling` modules lack initial module documentation.
- In the `two_step` module, the `WrappedKey` struct lacks documentation.
- The `u512` module, the `div_rem_u256` function documentation should state that when the overflow flag is set, the returned data for `quotient` and `remainder` is unusable.
- In `u8`, `u16`, `u32`, `u64`, `u128`, `u256` modules, the `mul_div` function documentation does not mention the abortion when the `denominator` is 0.
- In `u8`, `u16`, `u32`, `u64`, `u128`, `u256` modules, the `inv_mod` function documentation does not mention that when `value` is a multiple of `modulus`, it returns `none`.
- In `macros` module, the `mul_div_u256_fast` function's documentation only mentions the returned rounded quotient, ignoring the overflow flag.

Consider addressing the above instances of missing documentation to improve code readability and maintainability.

Update: *Resolved in pull request #175.*

N-05 Code and Documentation Inconsistencies

Throughout the codebase, multiple instances of code and documentation inconsistencies were identified:

- Most files do not follow a consistent format for function documentation. For instance, the `checked_shl` function of the `u8` module only describes the function behavior without specifying arguments and return values, whereas the `checked_shl` function includes these details along with a generics and aborts sections.
- In the `decimal_scaling` module, the return value documentation for the `safe_downcast_balance` and `safe_upcast_balance` functions lacks the `*`

character used in other modules. Furthermore, the `validate_decimals` function documentation lacks the `*` character in the abort section.
- In the `two_step` module, only the first error constant includes documentation, while subsequent constants do not. Likewise, the `delayed` module does not contain any documentation for its error constants.
- In the `macros` module, the first error constant lacks a trailing period in the error message, whereas all other error messages in the codebase include it.
- The `clz` and `log2` functions return `u16` in the `u256` module, since their maximum possible values (256) do not fit in `u8`, but return `u8` in all other modules. While technically correct, this creates an inconsistent public API where consumers must handle different return types depending on the integer width they are working with. The asymmetry is not immediately obvious from the function signatures alone, and switching from one integer width to `u256` requires updating all variable types that store these results.
- The `BIT_WIDTH` constant is declared as `u16` in the `u256` module (because 256 does not fit in `u8`) but as `u8` in all other modules (`u128`, `u64`, `u32`, `u16`, `u8`). This forces all sub-`u256` modules to cast `BIT_WIDTH` to `u16` when calling `macros` that expect a `u16` bit width. The inconsistency creates an asymmetry in the module API where `u256` has a unique type signature for its constant, and every other module uses a cast pattern.
- The documentation for `delayed::cancel_schedule` states that it "cancels the current transfer request if any". However, the function aborts when there is no pending transfer. The phrase "if any" may lead developers to assume the function handles the absence of a pending transfer. Consider updating the documentation to explicitly state that the function will abort/revert when there is no pending transfer.

Consider addressing the above-listed instances of code and documentation inconsistencies to improve the clarity and maintainability of the codebase.

Update: *Partially resolved in pull request #188.*

N-06 Incorrect Documentation

Throughout the codebase, multiple instances of incorrect documentation were identified:

- In the `u512` module, the documentation for the `msb` function states that it expects a non-zero value. Nevertheless, the function successfully returns `0` when a zero value is provided, instead of failing or aborting.

- In the `rounding` module, the `RoundingMode enum` documentation claims that rounding is only utilized within `mul_div` functions. However, rounding logic is used extensively across various other functions in the codebase.
- In `macros` module, the `mul_shr` function documentation states that the function will abort when the `shift` parameter is greater than or equal to 256. This is technically impossible as the `shift` parameter is of type `u8` which has a maximum value of 255.
- In the `macros` module, the documentation for the `round_log2_to_nearest` and `round_log256_to_nearest` functions indicate that they return a boolean value (`true` to round up, `false` otherwise). This is inaccurate because these functions return the final rounded integer value directly.
- The comment for the `average` macro describes it as "halving the difference with `mul_div_inner`". However, the actual code in line 36 calls `mul_div_u256_fast(delta, 1, 2, rounding_mode)` directly, bypassing `mul_div_inner` entirely.
- The module-level and `transfer` comments reference a function named `request_ownership`, but the actual function is named `request`.

Consider addressing the above-listed instances of incorrect documentation to improve the clarity and maintainability of the codebase.

Update: *Resolved in pull request #187.*

N-07 Inconsistent Error Handling in Mathematical Helper Functions

In the `inv_mod_extended_impl` function, the codebase exhibits mixed error-handling behaviors by utilizing both abort statements and returning `option::none()`. This inconsistency extends to the `mul_div` implementation within the unsigned packages. While `mul_div` is designed to return an `option` containing the result, the underlying helper functions, `mul_div_u256_wide` and `mul_div_u256_fast`, trigger an abort when the denominator is zero.

Furthermore, there is a functional discrepancy between the Move and Solidity implementations for the modular multiplicative inverse. In the Solidity version, providing a modulus of 0 returns 0 rather than reverting. In the context of the Solidity library, a return value of 0 is treated as the equivalent of Move's `option::none()`. Consequently, the current Move implementation, which aborts in this scenario, diverges from the established behavior of the Solidity counterpart. Inconsistent error handling can lead to unpredictable contract execution and

complicates the integration of these helper functions into parent logic, such as the `inv_mod` function.

Consider unifying the error-handling strategy within helper functions by consistently returning `option::none()` instead of aborting. This approach allows parent functions to handle empty results according to their specific requirements, ensuring greater flexibility and alignment with the Solidity implementation's logic.

Update: *Acknowledged, not resolved. The OpenZeppelin SUI development team stated:*

> *We believe that it makes sense to have two different approaches to error handling within the library. Passing divisor or modulus of 0 should be considered not as an expected use case but as an error. It is easy to guard against that issue by ensuring beforehand that the value is non-zero.*

N-08 Unhandled Overflow in `safe_downcast_balance`

The `safe_downcast_balance` function delegates to `scale_amount`, which computes `amount * 10^(target_decimals - source_decimals)` when scaling up. This multiplication can overflow `u256` before the u64 bounds check is reached, causing an abort with a generic arithmetic error instead of the intended `ESafeDowncastOverflowedInt`. For token balances, which is the expected use case for this library, the overflow is not reachable in practice. However, if the function is used outside of this context with large `u256` inputs, the overflow becomes possible.

Consider documenting the `u256` overflow boundary, or handling the scaling-up multiplication in a way that falls through to the `ESafeDowncastOverflowedInt` error for any input that would exceed `u64` after scaling.

Update: *Resolved in pull request #172.*

N-09 Multiple Evaluations of Macro Parameters

In the `checked_shl` and `checked_shr` macros, parameters prefixed with `$` are utilized directly within the macro body without being reassigned to local variables. This implementation is inconsistent with other macros in the library, such as `average`, `mul_div`, `mul_shr`, `inv_mod`, `mul_mod`, `log2`, `log256`, `log10`, and `sqrt`, all of which reassign their parameters to local variables before use.

When a macro expands, if a function call or an expression with side effects is passed as a parameter, that expression will be evaluated multiple times (once for every instance where the parameter appears in the macro body). This can lead to increased gas costs or inconsistent state if the passed function is not idempotent.

Consider reassigning all macro parameters to local variables at the beginning of the macro execution. This ensures that each argument is evaluated exactly once, maintaining symmetry with the rest of the codebase and preventing potential side effects during macro expansion.

Update: *Resolved in pull request #161.*

N-10 Unnecessary Option Destruction

In the `unwrap` function, the implementation includes an assertion to verify that the `recipient` variable is `none`. Subsequent to this check, the function calls `recipient.destroy_none()`. However, since the previous assertion already guarantees that the `recipient` is indeed `none`, the explicit call to destroy the empty `Option` is redundant. While this does not pose a security risk, it introduces unnecessary operations into the execution flow.

Consider removing the call to `destroy_none()` to simplify the codebase and improve readability.

Update: *Resolved in pull request #162.*

Conclusion

The OpenZeppelin SUI library provides a robust set of foundational modules aimed at enabling the development of secure, reusable, and reliable packages on the SUI blockchain. By mirroring the behavior and security guarantees of the widely adopted OpenZeppelin Solidity library while embracing the Move programming language and SUI's object-centric execution model, the library establishes a familiar yet purpose-built framework for developers building within the SUI ecosystem.

During the audit, one high-severity and two low-severity issues were identified. In addition, several opportunities to improve code clarity, consistency, and efficiency were noted. Overall, the codebase was found to be well structured and clearly documented, which significantly enhances auditability and supports straightforward adoption and integration by developers. As a general testing recommendation, ensuring that boundary and edge-case tests exercise all code paths in public-facing functions, not just in internal helpers, would further strengthen the reliability of the library. Additionally, care should be taken to ensure that test oracles remain independent of the logic under validation. When expected values are derived from the same logic that the test is meant to verify, problems in that logic can produce matching results on both sides, allowing tests to pass silently.

The OpenZeppelin development team is commended for their responsiveness and transparency throughout the review process. Their willingness to provide in-depth technical explanations, clarify architectural and design decisions, and share comprehensive documentation greatly facilitated the assessment and reflects a strong commitment to delivering secure and dependable infrastructure for the SUI ecosystem.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.